Host And Resource Safety

§ . . — Build-resource stats tracking mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "We need to incorporate through the Containers Submodule we use to run our System build Container the following mechanism safely: during its working lifetime we MUST track in proper Markdown document exported into PDF and HTML stats on System resources use! What was the peak and the minimum EVER the Container / System building process used! ... In top of the document we MUST have this ever values. ... After this we MUST present properly sorted divided by version tags names all build processes we were running with notes if the process completed with success or not, and resources usage during the process! Besides min and max values ever and per build iteration we MUST have for each resource we have average values — average memory usage, average CPU usage, and all other parameters IO and other resources!"

Classification: §11.4.17-classified **mixed** — the discipline of tracking host-side build-resource usage with min / max / mean / p95 per-build PLUS ever-values across all builds in a versioned Markdown + HTML + PDF triple is universal across every long-build project (AOSP, kernel, large monorepo, ML model training, multi-hour data pipelines). The implementation (registry path, monitor script name, exporter wiring) is project-specific.

The defect this anchor closes. Long builds (multi-hour AOSP builds, ML training runs, large monorepo CI) consume highly variable resource profiles over their lifetime — soong_build's 33 GB peak coexists with quiescent linker-only stretches at <2 GB. Without per-build sampling + ever-values, operators have NO empirical basis to (a) size containers correctly, (b) detect resource regressions across

versions, (c) prove which build iteration was the OOM-killer's trigger, (d) reason about parallelism caps (cf. §12.7 -j2 hard cap whose forensic basis would have been one full Stats.md generation earlier had this discipline been in place). Memory pressure debugging without time-series data is the bluff this anchor forbids.

The mandate. Every project under this Constitution with a build exceeding **1 minute wall-clock** MUST:

1. Run a host-side resource sampler for every build. The sampler runs as a sibling background process (not inside the build's own process tree), reads `/proc/meminfo` + `/proc/loadavg` + `/proc/stat`

- `/proc/diskstats` (or platform equivalents on non-Linux) at a fixed interval (recommended 5 s default — fine-grained enough to catch seconds-long peaks; coarse enough to keep itself <5% CPU and <50 MB RSS). Samples written as TSV.

2. Compute per-build summaries on stop. Per metric (memory used, CPU%, load average, disk read/write): **min, max, mean, p95**. p95 is the 95th-percentile sample value — required because mean alone hides extended-peak behaviour (a 2 h build with one 33 GB soong_build peak has mean ≈ 18 GB but p95 ≈ 32 GB; the latter is what operators must size for).

3. Append per-build summaries to a registry. One TSV row per build (build_id, status SUCCESS/FAIL/UNKNOWN, start/end timestamps, per-metric min/max/mean/p95, total disk read/write). The registry is the single source of truth — the Markdown report is derived.

4. Maintain ever-values across ALL builds. Min / max / mean across every tracked build, computed on every Markdown regeneration from the registry. Surface at the **top** of the report per User mandate verbatim.

5. Markdown + HTML + PDF triple. Stats.md (auto-generated), exported to Stats.html + Stats.pdf through the project's normal export pipeline. Triple stays in sync per §11.4.12. Committed via the project's lightweight doc-sync wrapper per §11.4.22.

6. Sort per-build entries by recency (most recent first) AND, where applicable, **group by version tag.** Operators reading the report want the latest build's profile at the top and the historical sweep underneath.

7. Track status per build. SUCCESS / FAIL / UNKNOWN — and if FAIL, capture the reason (exit code, OOM, ANR, etc.). A FAIL build's resource profile is the most operationally interesting one in the report — never silently drop FAIL entries.

8. Performance budget for the sampler itself. <50 MB RSS, <5% CPU. The monitor MUST NOT itself contribute to the resource pressure it is measuring (Heisenberg-class observer effect at scale). Pure /proc reads + awk arithmetic is the reference implementation; vmstat / pidstat acceptable; psrecord / heavy-Python-import samplers are forbidden.

9. Survive build failures. The stop hook MUST be called from both the success AND failure paths of the build wrapper so FAIL builds still produce a row. Hooking only the success path is a PASS-bluff at the telemetry layer.

Pre-build gate (recommended, per consuming project):

- `CM-BUILD-RESOURCE-STATS-TRACKER` — verifies (a) the monitor script exists + executable, (b) the Stats.md target file exists, (c) the build wrapper invokes start AND stop, (d) the doc-sync wrapper enumerates the Stats.{md,html,pdf} triple, (e) the exporter advertises the build-stats slug. Paired mutation hides the monitor script aside → gate FAILs.

Propagation. Composes with §11.4.12 (export-sync invariant — HTML + PDF in sync with Markdown), §11.4.18 (script-documentation discipline — the monitor ships with an external user guide), §11.4.22 (lightweight doc-sync commit wrapper enumerates the Stats.{md,html,pdf} triple), §12.6 / §12.7 / §12.9 (host-safety + containerized-build envelope whose forensic anchors are the empirical motivation for this telemetry).

No escape hatch. Build-resource tracking is not optional polish — it is the operational-evidence seam that converts post-mortem "the build OOM'd, no idea why" into "build X at git-SHA Y peaked at Z GB at minute N, which is W% above the rolling p95". The discipline pays for itself the first time it diagnoses a build-resource regression.

§ . . — Parallel-development methodology (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-19T~05:00Z MSK):

> "We MUST DO one comprehensive research and planning in the background in parallel with current mainstream work: our current methodolgy of the development is very slow. It takes us days to fix a bunch of bugs, add some changes or features and all this to be tested, verified and shipped. From iteration to iteration we are slower and slower. We MUST CREATE adjusted improoved version of working methodology where multiple workable items could be done in parallel, all come into the central (main) branch as they are done, then we at particular moment rebuild and reflash the System and in background full testing with validation and verification is done. Each parallel work on one workable (or more) item(s) must use parallel agents as much as possible! We MUST HAVE proper synchronization mechanism between parallel working routines so they do not cause conflicts, break features or create any other types of the problems! Document everything - the whole plan, whole methodology, working flow with in depth diagrams and graphs, all user guides and manuals and then add this all into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD! Make sure we start using this ASAP since we MUST increase efficiency and productivity a couple of times now! Writing in depth tests (all supported types of the tests) with Challenges and full HelixQA use is MANDATORY! Every test we execute besides executed with success MUST RESULT in proof that actual functionality being tested REALLY DOES WORK with NO BLUFF of any kind! Heavt enforcement of no-bluff / anti-bluff policy IS MANDATORY!"

Why this anchor exists. Sequential phase-chain working pattern (`Phase 39.A → Phase 39.B → … → Phase 39.DT` , observed 2026-05-05 to 2026-05-19: 86 unique Phase 39.X sub-phases across 14 days = ~6/day sub-phase rotation, 224 commits / 14 days = ~16/day) serialises on five distinct bottlenecks: (B1)

`commit_all.sh` flock at `.git/.commit_all.lock`, (B2) single-thread sub-phase chaining via `docs/CONTINUATION.md` hand-offs, (B3) rebuild-blocking on every `REQUIRES_REBUILD` fix (§12.7 -j2 cap = 4–6 h rebuild + 30 min flash

- 60 min validate per cycle), (B4) single-thread `test_all_fixes.sh` sweep (~30–60 min × 8 phase-device combinations), (B5) subagent dispatch latency (one-at-a-time spawn + wait pattern). Net effect: single-item wall-clock 1–4 days where productive work is < 4 h. The User mandate requires the methodology to multiply throughput several-x.

Operative rule. Project work proceeds through the Parallel Work Unit (PWU) pipeline rather than sequential phase-chain. Each PWU is a self-contained workable item with mandatory components (ATM-NNN identifier per §11.4.54, Issues.md entry per §11.4.15+§11.4.16, file- scope manifest, §11.4.43 RED test, source patch, pre-build gate per §11.4.4(b) layer 1, post-flash test per §11.4.4(b) layer 3, paired §1.1 meta-test mutation, §11.4.4(b) layer 4 HelixQA Challenge bank entry, captured-evidence directory per §11.4.5+§11.4.52). PWUs execute through five pipeline stages: **Stage 1 DEVELOP** (parallel, N PWU agents in isolated worktrees), **Stage 2 MERGE** (serial, conductor processes one PWU at a time via `commit_all.sh` flock + §11.4.41 4- step merge-first pipeline), **Stage 3 REBUILD+FLASH** (parallel where possible: AOSP build inside §12.7 bounded scope, D3+D4 flash serialised on USB hardware), **Stage 4 VALIDATE** (parallel on D3+D4+meta-test+ coverage-audit), **Stage 5 SWEEP** (parallel: HelixQA Challenge bank, Issues→Fixed migration, README.md doc-link refresh per §11.4.57, HTML+PDF exports). Stage 1 of round N+1 starts WHILE Stages 4+5 of round N are still running — this is the primary throughput multiplier.

Synchronization mechanism. Four-layer lock hierarchy: **L1** parent serial flock `.git/.commit_all.lock` (held only during Stage 2, released between PWUs), **L2** per-submodule git operations (implicit via cascade), **L3** contention-path advisory locks at `qa-results/pwu-locks/<sha256(path)>.lock` for the 10 forbidden cross- PWU paths (CLAUDE.md/AGENTS.md/Constitution.md/ CONTINUATION.md/ Issues.md/Fixed.md/pre_build_verification.sh/ meta_test_false_positive_ proof.sh/build.sh/commit_all.sh+push_all.sh/ device.mk+BoardConfig.mk/ atmosphere-*.sh/init.*.rc), **L4** per-PWU git worktree (zero cross- PWU file conflicts during Stage 1). Disjoint-scope PWUs (different test

files, different APKs, different drivers, different framework services) run fully parallel. Conflict detection at Stage 2 entry: `git fetch --all --prune --tags` + `git diff main...HEAD -- <scope>`

- cross-PWU scope overlap check. Overlap detected → PWU rejected to Stage 1 for rebase (per §11.4.41 step 2 integration-before-force-push).

Anti-bluff enforcement. Conductor REFUSES to merge a PWU lacking all four anti-bluff checks: **C1** §11.4.43 RED test captured-evidence (file exists + non-zero exit code in log + timestamp predates source patch — proves test catches regression), **C2** §1.1 paired meta-test mutation (applied + gate FAILs under mutation + restored + gate PASSes — proves gate is not a bluff gate), **C3** §11.4.50 deterministic- consistency (3 iterations for normal, 10 for cycle-validation, ALL identical exit codes AND identical evidence-hashes — eliminates §11.4.7 flake/ transient/intermittent demotion path), **C4** §11.4.5 captured-evidence (audio: WAV with non-trivial RMS + ffprobe channel count; video: screen recording with ffprobe frame count > 0 + analyzer matched event; UI: uiautomator dump with under-test element state; or HelixQA `result.json` PASS verdict + positive-evidence chain). HelixQA coverage is MANDATORY for every user-visible PWU per User mandate — Challenge bank entry in `tools/helixqa/banks/atmosphere.yaml` referencing the PWU's ATM-NNN + dispatching to the on-device test + scoring PASS only on positive captured evidence. Metadata-only PASS / configuration- only PASS / absence-of-error PASS / grep-without-runtime-evidence PASS all REJECTED at the merge gate.

Agent orchestration. Four roles: **Conductor** (1, main thread) — issues ATM-NNN, dispatches PWU agents, runs Stage 2 merge serially, runs Stages 3-5, owns all contention-path edits, maintains `docs/CONTINUATION.md` §3 live snapshot. **PWU agent** (4–6 background) — one PWU each, Stage 1 work in isolated worktree, signals `qa-results/pwu-<atm>/READY_FOR_MERGE` on completion. **Validator** (4 background) — Stage 4 work in parallel on D3+D4+meta-test+coverage- audit. **Sweeper** (2 background) — Stage 5 work in parallel with next round's Stage 1. Max concurrent agent count ~12 at peak, bounded by §12.6 60% memory budget (~6 GB total ≤ 38 GB budget). Composition with §12 host-session safety: conductor refuses dispatch if `host_check_safety` fails; per §12.7 AOSP build still hard-capped at -j2 inside bounded scope; per §12.8 + §12.8b + §12.8c concurrency gates still block on stray gradle/kotlin daemons.

Composition map (mandatory):

- §11.4.4 — four-layer test coverage per PWU (pre-build + post-build + on-device + HelixQA bank).
- §11.4.5 — audio/video quality analysis comprehensiveness (PWU captured-evidence directory).
- §11.4.6 — no-guessing mandate (every PWU claim backed by captured forensic evidence or explicit `UNCONFIRMED:` tag).
- §11.4.7 — demotion-evidence rule (3/3 deterministic-consistency closes the demotion-bluff path).
- §11.4.9 — batch-source-fixes-before-rebuild (Stage 2 → Stage 3 batch threshold).
- §11.4.15 + §11.4.16 + §11.4.33 — PWU status + type vocabulary + type-aware closure terminal value.
- §11.4.19 — atomic Issues.md → Fixed.md migration on PWU closure.
- §11.4.41 — Pre-Force-Push Merge-First 4-step pipeline at Stage 2 entry.
- §11.4.42 — Iteration-discipline (PWU pipeline IS the iteration conductor; §11.4.58 binds steps 1-5 to PWU stages 1-5).
- §11.4.43 — TDD RED-first per-PWU enforced at merge time.
- §11.4.45 — Integration-status-doc maintenance (PWU's Status.md updated at every stage transition).
- §11.4.49 — Dual-approach testing (PWU MUST ship both UI-driven AND Intent/Broadcast-driven variants when applicable).
- §11.4.50 — Deterministic-consistency (3-iter normal / 10-iter cycle-validation merge-time enforcement).
- §11.4.52 — Autonomous-validation (PWU's on-device test MUST run without operator presence per Stage 4 parallel design).
- §11.4.54 — ATM-NNN ticket identifier (PWU identifier replaces Phase 39.X letters).
- §11.4.57 — README.md doc-link refresh at Stage 5.
- §12.6 — 60% memory-budget ceiling (caps concurrent agent count).
- §12.7 — AOSP `m -j` hard-cap at -j2 (Stage 3 build cap).
- §12.8 + §12.8b + §12.8c — concurrency hardening (Stage 3 preflight refuses on running gradle/kotlin/git-pack-objects/load-per-cpu spike).

- §12.10 — CONTINUATION.md maintenance (conductor owns the file by §3.4 forbidden cross-PWU rule).
- §9.2 — data safety (every history rewrite during Stage 2 merge-first step 2 backed by hardlinked `.git` backup).

Pre-build gates:

- `CM-PWU-LOCK-HIERARCHY` — verify lock hierarchy script exists at `scripts/testing/pwu_acquire_path_lock.sh` and `scripts/testing/pwu_release_path_lock.sh`, both executable, both reference the 10 forbidden cross-PWU paths from §3.4 of `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.
- `CM-PWU-ANTI-BLUFF-COVERAGE` — for every PWU with status "Ready for merge" or beyond, assert all four anti-bluff checks (C1–C4) have captured-evidence files under `qa-results/pwu-<atm>/evidence/`. PWU lacking any of red-test-output / mutation_patch / deterministic- consistency / captured-feature-proof FAILs the gate.
- `CM-PWU-MERGE-QUEUE-DISCIPLINE` — scan `qa-results/pwu-queue/` for PWUs that bypassed the merge queue (commits on main without a corresponding `qa-results/pwu-queue/merged/<atm>` marker). Any bypass FAILs the gate.
- `CM-PWU-PARALLEL-AGENT-LIMIT` — assert no more than 6 concurrent background PWU agents (count tmux/screen sessions or worktree branches) — over-limit FAILs the gate (per §12.6 memory budget).
- `CM-COVENANT-114-58-PROPAGATION` — anchor literal `§11.4.58` present in `constitution/CLAUDE.md` + `constitution/AGENTS.md` + parent CLAUDE.md/AGENTS.md + every owned-submodule CLAUDE.md/AGENTS.md (42 files total in the current consuming project's family).

Paired mutations (per §1.1):

- Move `scripts/testing/pwu_acquire_path_lock.sh` aside → `CM-PWU-LOCK-HIERARCHY` FAILs.
- Touch a PWU's `READY_FOR_MERGE` marker without `red-test-output.log` → `CM-PWU-ANTI-BLUFF-COVERAGE` FAILs.
- Land a commit on main without a corresponding `qa-results/pwu-queue/merged/<atm>` marker → `CM-PWU-MERGE-QUEUE-DISCIPLINE` FAILs.

- Spawn a 7th concurrent PWU agent → `CM-PWU-PARALLEL-AGENT-LIMIT` FAILs.
- Strip `§11.4.58` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-58-PROPAGATION` FAILs.

No escape hatch. No `--skip-merge-queue`, `--allow-bypass`, `--no-anti-bluff-check`, `--unlimited-agents`, `--sequential-phase- chain-mode` flag exists in any pipeline helper. The discipline exists because (a) the User mandate explicitly requires the multiplier ASAP and (b) the §11.4 covenant specifically prohibits PASS-bluffs (which the four anti-bluff merge-time checks mechanically prevent). Operators who feel they need to bypass the pipeline for a specific high-urgency fix should split the work into a single trivial PWU and let the pipeline run normally — Stage 1 of a trivial PWU is < 30 min, Stage 2 is < 5 min, Stage 3 is the same rebuild cost they would pay anyway.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution that has multiple owned-submodules

- a sequential commit/push tool + a captured-evidence testing discipline. Project-specific implementations (agent dispatch helpers, worktree layout, batch thresholds) live in consumer-side `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md` (this Constitution defines the discipline; consumers implement the tooling).

Reference: parent `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md` (full methodology with diagrams, templates, operator manual, and migration plan).

2026 05 27

§ . . — Safe-parallel-work-with-long-build catalogue + mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-27):

> "Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? Anything workable which will not affect the build process or final image we generate for the flashin. If yes, and that was not already clear by our root Constitution, we should add all mandatory details into it ... Make sure we do as much as possible work in background in parallel with main work stream and oreferrably using subagents-driven approach!"

§11.4.94 mandates "always check parallel-work feasibility before idle"; §11.4.89 mandates background-test discipline; §11.4.58 defines the parallel-work-unit pipeline. §11.4.96 is the **operational catalogue** that enumerates, for the canonical long-running workload (5-7 h AOSP containerised build per §12.9), exactly what CAN safely run in parallel and what MUST NOT — closing the ambiguity gap §11.4.94 left implicit.

Closed-set categories — SAFE during AOSP containerised build:

- **(A) Markdown / documentation work** anywhere under `docs/`, `*.md`, README, CLAUDE.md, AGENTS.md, QWEN.md, Status.md / Status_Summary.md, Issues.md / Fixed.md / their Summary siblings, changelogs, research notes. The build container mounts source read-only at a snapshot moment + reads only the build-relevant files (Android.bp / Android.mk / source code / vendor binaries). MD edits do NOT propagate into the build's read scope.
- **(B) Generator + helper script work** under `scripts/`, `scripts/testing/`, `scripts/llm/`, `scripts/firebase/`, etc. — these are host-side helpers NEVER consumed by the AOSP build. Safe to edit + commit + push freely.
- **(C) Pre-build / meta-test gate authoring + paired §1.1 mutations** under `device/rockchip/rk3588/tests/pre_build_verification.sh` + `scripts/testing/meta_test_false_positive_proof.sh` — these run host-side via `bash pre_build_verification.sh`; the AOSP build itself does NOT execute them inline. Safe to edit during build; the NEXT pre-build run (post-current-build) picks up the new gates.

- **(D) On-device tests** under `device/rockchip/rk3588/tests/test_*.sh` — these run on D3/D4 via ADB post-flash. Authoring new test scripts during build does NOT affect the build (the build packages the test corpus from the source tree but does so once at copy-out time; scripts added after the copy-out are absent from THIS image but present in source for the NEXT image).
- **(E) Constitution submodule edits + push** — separate git workspace at `constitution/`, not on the AOSP build's path. Edits + push to all 6 constitution remotes safe at any time.
- **(F) Project submodule commit + push** to either the parent or owned submodules' remotes — per §11.4.88 background push, this is OS-level orthogonal to the container.
- **(G) D3 / D4 live-ADB probes** (read-only `dumpsys` / `getprop` / `cat /proc/asound/...` / `screencap` / `logcat -d`) — these query device state without changing it; do NOT affect the host build.
- **(H) Subagent dispatch** per §11.4.20 / §11.4.70 — each subagent runs in isolated context window; multiple subagents in same checkout coordinate via §11.4.84 working-tree quiescence (lockfile or git worktree per disjoint scope). Read-only analysis subagents (obsolescence audit, summary clarity sweep, code-review) are uniformly safe.
- **(I) Web research + external API queries** (with §11.4.10 credential discipline preserved) — operator-relevant browsing, ASUS QVL lookups, vendor spec verification, package-registry version checks.
- **(J) Workable-items DB operations** per §11.4.93 + §11.4.95 — `workable-items sync` (when implemented), `workable-items validate`, `workable-items report` — read/write SQLite at `docs/workable_items.db`, orthogonal to build.
- **(K) Pre-build verification + meta-test execution** — RUN per §11.4.89 background-test discipline. The build container reads source; pre-build runs host-side reading the same source. No conflict at filesystem layer (read-only intersection).

Closed-set — UNSAFE during AOSP containerised build:

- **(α)** **`git checkout`** / **`git reset --hard`** / **`git clean -df`** on the source tree — would change files the build is reading; introduces inconsistency. Defer until build completes OR use a `git worktree` per §11.4.84.

- **(β) Mass file deletions or renames** under `device/`, `frameworks/`, `hardware/`, `vendor/`, `kernel-5.10/`, `external/`, `packages/`, `bionic/`, `bootable/`, `build/`, `system/`, `cts/`, `tools/`, `prebuilts/`, etc. — anything the AOSP build potentially reads. Apply ONLY after build completes.
- **(γ) Submodule pointer updates that affect built APKs** — bumping `device/rockchip/atmosphere/presenter` or `smarttube-player` or `vlc-player` etc. pointer changes the binary content the build packages. Defer pointer bumps to between-build windows OR plan for the NEXT build to pick them up.
- **(δ) `out/` directory mutations** — the build's output directory. Never touch concurrently.
- **(ε) `make clean` / `m clobber` / `rm -rf out/`** while build runs — direct conflict; will likely crash the build.
- **(ζ) Container destruction** (`podman stop atmosphere-aosp-build`, `podman pod rm`) — terminates the build mid-flight. Operator-explicit only.
- **(η) Disk-filling operations** (large file downloads, container image builds, NVMe duplication) that push the host below the §12.9 free-space minimum — could OOM the build's writable layer.
- **(θ) Host-session-safety breaches** per §12 — suspend / hibernate / logout / unbounded memory ops in user.slice — kill the build's parent cgroup.

Conductor responsibility (per §11.4.94 already; §11.4.96 makes the catalogue explicit):

- Before EVERY pause point during a long build, the conductor MUST consult this catalogue, identify all (A)-(K) items in the priority queue per §11.4.42 + §11.4.72, dispatch at least one per §11.4.20 / §11.4.70 subagent-driven default + §11.4.89 background-task discipline. "Build is running, nothing else to do" is NEVER true per §11.4.94 + this catalogue.

Subagent-driven default for catalogue items:

- (A) MD work + (E) constitution edits + (F) commits: conductor-direct (lightweight + critical-state sequencing).
- (B) generator/helper work + (C) gate authoring + (D) on-device test authoring + (G) live-ADB probes + (I) web research + (J) DB ops: subagent-dispatchable per §11.4.20.

- (H) is itself the subagent dispatch — composes with the rest.
- (K) pre-build / meta-test execution: backgrounded per §11.4.89, conductor polls outcome.

Pre-build gate `CM-COVENANT-114-96-PROPAGATION` enforces this anchor literal across the canonical fleet. Pre-build gate `CM-PARALLEL-WORK-DURING-BUILD-AUDIT` (when implemented) audits recent commits during AOSP build windows for the parallel-work output (commits landing during a build window with timestamps inside `qa-results/aosp_build_*.log` start/end). Paired §1.1 meta-test mutations strip the catalogue literals → gates FAIL.

Composes with §11.4.20 (subagent-driven) + §11.4.42 (iteration priority) + §11.4.58 (parallel PWU) + §11.4.70 (subagent default) + §11.4.72 (audio top-priority) + §11.4.82 (iteration-speedup) + §11.4.84 (working-tree quiescence — esp. for subagent parallelism) + §11.4.85 (stress + chaos can run in parallel) + §11.4.87 (endless-loop zero-idle) + §11.4.88 (background-push) + §11.4.89 (background-test) + §11.4.92 (multi-pass evaluation per parallel branch) + §11.4.93 / §11.4.95 (workable-items DB) + §11.4.94 (parallel-work survey mandate) + §12.6 / §12.7 / §12.8 / §12.9 (host-session safety + build cap).

Canonical authority: this Constitution.md §11.4.96 in the HelixConstitution submodule.

Non-compliance is a release blocker. Conductor scheduling a wake/sleep during an active AOSP build without first dispatching at least one (A)-(K) catalogue item from the priority queue is a §11.4.94 + §11.4.96 violation = §11.4 PASS-bluff at the operating-mode layer.

2026 06 03

§ . . — Resolve-by-stable-name-not-by-enumeration-index mandate (research-derived, - -)

Short tag: `resolve-by-stable-name` .

Forensic anchor (genericised, 2026-06-03). A platform bound an audio output to a kernel-enumerated device index (`card=0`). A second device of a different class enumerated FIRST at boot and took slot 0, shifting the intended device to slot 1. The static index binding now pointed at the wrong device: the policy layer failed to

attach the intended sink, mis-assigned its TYPE, and the user-facing output switcher labelled the AV receiver "Wired Headphone" while routing collapsed to stereo. The lower layer of the SAME stack already resolved the device correctly **by name** (scanning `/proc/asound/card*/id` for the controller name) — proving the brittleness was the *index* binding, not the resolution capability. The defect class generalises to every enumerated resource whose ordinal is assigned at discovery/boot/hotplug time and is therefore non-deterministic across reboots, device additions, and topology changes.

The mandate. Any binding between a configuration / policy / code site and a hardware device, resource handle, or enumerated entity — audio cards, display connectors, network interfaces, storage devices, GPU render nodes, input devices, camera indices, container/process slots, any registry whose members are assigned an ordinal at discovery time — MUST resolve the target by a **stable identifier** (name, UUID, serial, label, controller-name, content-derived hash, sink-reported identity) and MUST NOT resolve it by an enumeration index / ordinal / slot number, UNLESS the platform documents that ordinal as deterministically pinned (and the pinning mechanism is itself part of the binding, captured and asserted). Where a stable identifier exists at one layer of a stack, every other layer binding the same resource MUST use the same identifier — mixed by-name-here / by-index-there is the structural weak link this anchor forbids. Where the platform offers a sink-reported / capability-derived identity (e.g. an EDID/ELD-style capability descriptor), preferring it makes the binding robust to the broadest class of topology change.

Honest boundary. When the platform genuinely provides no stable identifier and only an ordinal exists, the binding MUST (a) pin the ordinal deterministically via the platform's own pinning mechanism, (b) capture that pin as part of the binding's runtime signature per §11.4.108, and (c) document the residual fragility as `UNCONFIRMED:`-class risk per §11.4.6 — never silently trust an unpinned ordinal.

Classification: universal (§11.4.17) — index-vs-name resource resolution is a platform-neutral binding-robustness discipline reusable by ANY project that binds to enumerated hardware / resources / handles; the consuming project supplies its concrete stable-identifier mechanism (ALSA card name, DRM connector name, NIC predictable name, block-device UUID, etc.) and the specific layers that must agree per §11.4.35.

Composes with §11.4.6 (no-guessing — "the index is *usually* stable" is the exact guess this anchor forbids; prove the pin or bind by name), §11.4.8 (deep-web-research — mature platform stacks resolve by name/UUID; reproducing a known-brittle index binding when the by-name path is documented is a §11.4.8 violation by omission), §11.4.69 (sink-side positive-evidence — the by-name binding's correctness is verified by the downstream/sink-reported identity, not by the local config), §11.4.108 (four-layer fix-verification — the resolved-by-name binding's runtime signature is asserted on a clean target across the topology that originally broke the index binding), §11.4.110 (pre-build clash detection — a new ordinal-based binding in a diff is a statically-catchable clash class the change-impact detector flags).

Propagation. Propagation gate `CM-COVENANT-114-111-PROPAGATION` enforces the literal anchor `11.4.111` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-RESOLVE-BY-NAME-NOT-INDEX` (diff-driven scan flagging a new enumeration-index binding to a resource that exposes a stable identifier, unless an accompanying deterministic-pin + runtime-signature is present); paired §1.1 mutation plants an unpinned index binding → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.111 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--allow-index-binding`, `--ordinal-is-stable-enough`, `--skip-name-resolution`, `--trust-unpinned-index` flag exists.

2026-06-03

§11.4.112 — Single-resource-owner partitioning for parallel hardware testing mandate (anti-bluff-dev remediation, 2026-06-03)

Short tag: `single-resource-owner-partitioning`.

Forensic anchor (genericised, 2026-06-03). Multiple parallel test/discovery streams ran against shared physical hardware (devices D3 + D4, an HDMI audio sink). Two streams driving media playback on the SAME device at once corrupt each other's evidence — overlapping audio routes the sink reports the wrong

codec, concurrent `am start` fights over the foreground, a second stream's input events land in the first stream's app. The remediation enforced: exactly ONE stream OWNS each device's exclusive/media resource at a time; other streams targeting that device are read-only (probes / dumpsys / log capture). Parallelism is partitioned BY DEVICE — stream A owns D3's media, stream B owns D4's media — so the parallel-development pipeline (§11.4.58 / §11.4.103) runs without the streams clobbering each other's captured evidence.

The mandate. When multiple parallel work/test/discovery streams exercise SHARED hardware or any exclusive-access resource (a media-playback path, a single HDMI/audio sink, an exclusive device handle, a single serial/JTAG line, a GPU under exclusive capture), exactly ONE stream MUST own each such resource at a time. The exclusive owner drives the resource (playback, input injection, capture); every other concurrent stream targeting the same resource MUST be READ-ONLY (passive probes — `dumpsys` / `/proc` / `/sys` reads / sink-side network probes / log tails) for the duration. Parallelism MUST be partitioned by resource: distinct devices / sinks / handles run fully concurrent (stream-per-device), but the same device's exclusive resource is single-owner. The ownership MUST be enforced by an advisory lock / token (per §11.4.58 L3 contention-path locking + §11.4.84 working-tree quiescence's hardware analogue), not by convention — a second stream MUST be unable to silently start media on a device another stream owns. Ownership is event-driven: a stream claims a device when its exclusive resource frees and releases it the moment its work completes, so the next queued stream can claim it (per §11.4.103 auto-backfill).

Why single-owner. Concurrent drivers of one exclusive resource produce CROSS-CONTAMINATED evidence: the sink reports whichever stream's audio won the race, the foreground belongs to whichever `am start` landed last, input events interleave. A PASS captured under contention is a §11.4 evidence-integrity bluff — the captured artefact does not reflect what the test under inspection actually did. Single-owner partitioning is the precondition that makes per-stream captured evidence trustworthy under parallelism.

Honest boundary (§11.4.6). A read-only probe stream is genuinely read-only — it MUST NOT issue any state-changing command (no `am start`, no input injection, no setprop, no playback) against a device it does not own; a "read-only" stream that

mutates state is the contention the partition forbids. When the resource genuinely cannot be partitioned (a single device the whole batch needs to drive), the streams MUST serialize on it (single-owner over time), not run concurrently and hope.

Classification: universal (§11.4.17) — single-owner-per-exclusive-resource partitioning is a platform-neutral parallel-testing discipline reusable by ANY project with parallel streams contending over shared hardware / exclusive handles; the consuming project supplies its resource inventory (which devices / sinks / handles are exclusive) and lock mechanism per §11.4.35. Strict refinement of §11.4.58 / §11.4.103 for the hardware-contention case.

Composes with §11.4.5 / §11.4.69 (captured-evidence integrity — single-owner is the precondition for trustworthy per-stream evidence), §11.4.13 (sink-side evidence — a shared sink is the canonical single-owner resource), §11.4.50 (deterministic consistency — contended evidence is non-deterministic, defeating N-iteration stability), §11.4.58 (parallel-development PWU — §11.4.119 is the hardware-resource refinement of its L3 contention-path locking), §11.4.82(F) (parallel multi-device testing — per-device streams ARE the partition), §11.4.84 (working-tree quiescence — the hardware analogue: a stream owns the device's exclusive state cleanly before it drives it), §11.4.103 (continuous parallel streams — single-owner + event-driven release enables the auto-backfill), §11.4.118 (discovery pressure — parallel per-device discovery needs this partition).

Propagation. Propagation gate `CM-COVENANT-114-119-PROPAGATION` enforces the literal anchor `11.4.119` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-SINGLE-RESOURCE-OWNER-PARTITION` (parallel hardware-test streams acquire a per-resource ownership lock before any state-changing command; concurrent same-resource streams are read-only); paired §1.1 mutation removes the ownership-lock acquisition (letting two streams drive one device) → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.119 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--allow-concurrent-resource-drivers`, `--skip-ownership-lock`, `--read-only-may-mutate`, `--contended-evidence-OK` flag exists.

§ . . — Always-on device-recording mandate (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-06): we MUST ALWAYS live-record all available data from all devices we use for testing or that we know manual testing is being performed on, done EXTRA carefully so the recording never harms the device, its performance, or causes side effects; recorded raw data MUST NOT be processed without need (token-conscious) and is ALWAYS excluded from version control + the code-intelligence index; only curated analysis/evidence is committed, and only during release preparation.

The mandate (ALL must hold). For EVERY test/debug device the project uses — and every device on which the operator is known to be performing manual testing — across EVERY reachable transport (USB or wireless ADB / SSH / serial / network introspection API), the project MUST ALWAYS live-record all available data sources it can later analyse: all activities, all logs (system log, kernel log, dropbox, crash/ANR traces), performance metrics (CPU / memory / I/O / thermal / load), every sink-side / downstream introspection report per §11.4.13, and any other live-changeable parameter. (1) **Extra-careful, side-effect-free.** Recording MUST be designed so it NEVER harms the device, degrades its performance, or causes a side effect — non-invasive read-only probes only, bounded sampling intervals, bounded write-volume, an observer-effect budget (no probe that mutates the system-under-test). A recorder that perturbs the device-under-test is a §11.4.128 violation, NOT evidence. (2) **Background + parallel + subagent-driven.** Recording MUST run in the background in parallel with the main work stream (per §11.4.103 continuous-parallel-stream), dispatched subagent-driven per §11.4.70 — never blocking the main stream, never the only thing the conductor does. (3) **Token-conscious — record-now, analyse-later.** Raw recorded data MUST NOT be processed / analysed without need; the default is capture-and-store. The ONLY standing exception is during release-tag preparation (the §11.4.40 full-suite retest / §11.4.42 release-gate) OR when the operator explicitly asks — at which point the relevant slice is analysed and the curated findings become captured-evidence. Analysing raw recordings continuously, eagerly, for no tracked reason burns tokens and is a §11.4.128 violation. (4) **Raw is git-ignored AND code-intelligence-excluded; only curated evidence is committed.** The raw recording corpus MUST be excluded from version control (with a §11.4.77 regeneration-mechanism

declaration — the mechanism is "re-record by re-running the recorder against the device", with the honest caveat that a past device-state is not byte-reproducible) AND excluded from the code-intelligence index per §11.4.78/§11.4.79 (a raw log/recording corpus is not source; indexing it pollutes the graph). Only the CURATED analysis / extracted evidence artefacts are committed — and only during release preparation per §11.4.83 (`docs/qa/<run-id>/`) — never the raw corpus. (5) **Deterministic archive layout.** The raw corpus MUST be laid out by date → combined-source-state-hash → device → recording-sequence so any artefact is traceable to the exact code state and device it came from: `<recording-root>/YYYY-MM-DD/<combined main+submodules state hash>/<DEVICE>_<SERIAL>/recording_NNN/<files>` (monotonic `NNN` per device per state-hash). (6) **Anti-bluff.** A recorder claimed running but producing no growing corpus is a §11.4 bluff; a curated finding cited as evidence MUST trace to a real raw-corpus path under the deterministic layout; the recorder's own health (alive + corpus growing) is itself a captured-evidence assertion per §11.4.5/§11.4.69.

Classification: universal (§11.4.17) — a platform-neutral test-evidence-capture discipline reusable by ANY project with reachable test/debug devices; the consuming project supplies its concrete device serials, transport probes, recorder helper, recording-root, and performance-metric set per §11.4.35. Composes with §11.4.2 (recorded-evidence — §11.4.128 guarantees the recording exists to BE the evidence) / §11.4.5 (captured-evidence quality) / §11.4.13 (sink-side introspection IS one recorded source) / §11.4.69 (sink-side positive-evidence taxonomy) / §11.4.40 + §11.4.42 (release-prep is the analyse-the-corpus trigger) / §11.4.70 (subagent-driven recorder dispatch) / §11.4.77 (raw corpus git-ignored WITH a regeneration-mechanism declaration) / §11.4.78 + §11.4.79 (raw corpus excluded from the code-intelligence index) / §11.4.83 (curated evidence lands under `docs/qa/<run-id>/`) / §11.4.103 (background parallel stream) / §11.4.119 (single-resource-owner — a read-only recorder never contends with an exclusive driver). Propagation gate `CM-COVENANT-114-128-PROPAGATION` (literal `11.4.128` across the consumer fleet) + recommended gate `CM-DEVICE-RECORDING-ALWAYS-ON` (recorder present + raw-corpus git-ignored + code-intelligence-excluded + deterministic layout) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: this Constitution.md §11.4.128 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-recording`, `--record-without-layout`, `--commit-raw-corpus`, `--index-raw-corpus`, `--analyse-corpus-always`, `--invasive-probe-OK` flag exists.

§ . . — Tracked/recorded-device availability-following mandate (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-10): every device the project is tracking / following / recording MUST be availability-followed with the project's already-defined reconnection timings + the sanctioned device-recovery path; a silent recording/tracking gap — papering a disconnect as continuous capture — is a §11.4 bluff.

Every device the project is recording or tracking (test / debug / manual-testing device, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED: its connection state continuously monitored, and any drop handled — never silently abandoned and never presented as a continuous recording. The always-on recorder under §11.4.128 KNOWS a tracked device is absent (its per-device loop guards on the reachable state) but, lacking a following discipline, merely spins idle — no data captured, no offline event logged, no resume, no escalation — so the recording corpus presents a continuous timeline with a silent hole. That silent hole is a §11.4 PASS-bluff at the recording-integrity layer: it claims continuous capture while no data exists for the gap.

On a tracked device leaving its reachable state the system MUST, automatically: (a) **DETECT** the drop and **log an honest offline event** into the recording corpus (§11.4.6 — a silent gap presented as continuous capture is a fabricated-continuity bluff; §11.4.128 — a silent recording gap defeats always-on recording); (b) **WAIT** for the device to return using the project's ALREADY-DEFINED reconnection timings — never invented numbers (§11.4.6), the SAME grace / reconnect / poll budgets the project's recovery path already uses; (c) **RE-ATTACH** — resume recording / tracking the moment the device returns and log an honest online / resume event; and (d) **ESCALATE** to the project's sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY,

never bypassing its gate and never performing a destructive recovery (e.g. a power-cycle) autonomously without that authorization (§11.4.21 / §11.4.101 — high-blast-radius recovery is gated; while blocked the system keeps following the device, logging the blocked-escalation honestly). A tracked-device drop that produces a silent corpus hole, a never-resumed recording, or a never-escalated permanent absence is the bluff this anchor forbids.

Classification: universal (§11.4.17) — a platform-neutral device-following discipline reusable by ANY project that records or tracks devices; the consuming project supplies its concrete tracking transport, the device set, the already-defined reconnection timings, and the sanctioned recovery entry point per §11.4.35. Composes §11.4.128 (always-on recording — §11.4.144 closes its drop-handling gap) / §11.4.69 (`device_recovery` sink-side positive evidence) / §11.4.6 (honest offline / online events, reused-not-invented timings) / §11.4.14 (watchdog children reaped on stop) / §11.4.21 + §11.4.101 (gated, non-autonomous escalation). Propagation gate `CM-COVENANT-114-144-PROPAGATION` (literal `11.4.144` across the consumer fleet) + recommended gate `CM-DEVICE-AVAILABILITY-FOLLOWED` (the recorder wires a per-device availability watchdog that reuses the defined timings + escalates only through the gated recovery path) + paired §1.1 meta-test mutation (strip the watchdog wiring / let an absence go unlogged → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.144. Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-availability-following`, `--silent-recording-gap-OK`, `--no-reconnect-wait`, `--invent-reconnect-timing`, `--skip-recovery-escalation`, `--autonomous-power-cycle-OK` flag exists.

2026 06 10

§ . . — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, - -)

Forensic anchor — verbatim operator intent (2026-06-10): "any agent that crashed because of something MUST BE respawned and finish its work at some point! We MUST NOT lose any work, forget about or have it corrupted!" Forensic case study (FACT, this session): dispatched subagents died on **transient** causes —

`API Error: Server is temporarily limiting requests` (rate-limit) killed 5 at once, `API Error: socket connection closed unexpectedly` killed 2, and one left PARTIAL edits (two source files written, the dependent SystemUI sliders + doc + test NOT done). Each was re-dispatched by pure conductor vigilance with **no mechanical guarantee** — the moment conductor context is lost / compacted / the conductor itself crashes, an in-flight-but-dead work unit is silently dropped: work lost / forgotten / corrupted.

Every agent/subagent dispatched to perform a work unit MUST be tracked through its full lifecycle so that a crash NEVER loses, forgets, or corrupts its work. A crashed agent is **NOT** a completed agent (§11.4.6 — "it probably finished before dying" is a forbidden guess; an abnormal termination is positive evidence the work unit is OPEN). The mandate (ALL hold):

(a) Mandatory work-unit / agent REGISTRY. Every dispatched agent/subagent MUST have a durable, append-only, machine-readable registry entry carrying at minimum: a stable agent/work-unit id, its task + file-scope (§11.4.58 PWU manifest), its declared output path(s) + captured-evidence dir (§11.4.5/§11.4.69), its dispatch timestamp, and a status from the CLOSED SET `{dispatched | in-flight | crashed | respawned | complete}`. The registry reuses the §11.4.116 sync substrate (append-only JSONL event stream + atomically-rewritten status snapshot) — agent-lifecycle transitions are events; the §11.4.116 integrity property holds ("an entry with no dispatch-event cannot show `complete`"). The registry is the SINGLE SOURCE OF TRUTH for "is this work owed?" — an unregistered dispatch is itself a §11.4.147 violation (an untracked agent cannot be respawned).

(b) Mechanical CRASH-DETECTION + RESPAWN-until-complete. Any abnormal agent termination — transient API/rate-limit error (`Server is temporarily limiting requests`), socket-close (`socket connection closed unexpectedly`), any non-completion exit, OR a terminal error after the runtime's own retries are exhausted — MUST flip the registry entry to `crashed` and keep the work unit OPEN. An OPEN unit MUST be respawned (a fresh agent claims the same id + scope + preserved state) and re-respawned until an agent reaches `complete`. Respawn is the safe, reversible, bounded-blast-radius decision the conductor takes autonomously per §11.4.101 (never blocks the loop waiting for a human). Transient causes use the project's ALREADY-DEFINED backoff/retry

budgets — never invented numbers (§11.4.6); a genuinely-non-transient terminal error (deterministically reproducible after respawn) is investigated per §11.4.102 before further respawn, never spun in a blind retry loop.

(c) PARTIAL-STATE handling — preserve, then resume-or-clean-restart. A crashed agent's uncommitted edits + output doc MUST be PRESERVED (never silently discarded → work lost; never blindly committed → corruption). Before resuming, the respawn MUST run the §11.4.84 quiescence check on the preserved tree (every modified file accounted-for against the unit's scope; no mutation/ `// always pass` / `_mutated_*` residue; no half-written/torn artifact). Then EITHER (i) **RESUME idempotently** from the preserved partial when it passes the check (consistent + complete-enough to continue), OR (ii) **CLEAN-RESTART** the unit from a known-good base (§9.2 pre-op backup, reversible per §11.4.101) when the partial is inconsistent / incomplete-unsafely / fails the §11.4.84 check. Either path guarantees nothing is lost and nothing is corrupted. Per-agent `git worktree` isolation (§11.4.58 L4 / §11.4.84) keeps a crashed unit's partial tree from contaminating other streams.

(d) COMPLETION criterion. A work unit is DONE **only** when an agent reaches `complete` with its required captured evidence/output landed (§11.4.5/§11.4.69 + the §11.4.116 verdict-carries-evidence-path rule). The endless-loop done-condition (§11.4.87 / §11.4.94 / §11.4.97 / §11.4.126) MUST NOT read as satisfied while any registry entry is `dispatched` / `in-flight` / `crashed` / `respawned`-not-yet-`complete`; the zero-idle survey (§11.4.94) MUST treat every non-`complete` entry as an OPEN actionable item to reclaim. A registry showing `complete` without the landed evidence is a §11.4 PASS-bluff at the agent-lifecycle layer.

Honest boundary (§11.4.6): the registry + respawn guarantee that work is *not lost or corrupted*; it does NOT itself prove the work is *correct* — the respawned unit's output still crosses §11.4.108 four-layer verification, the §11.4.125/§11.4.142 code-review gate, and §11.4.40 full-suite retest. §11.4.147 is the durability layer beneath those, not a substitute. §11.4.147 is the agent-side analogue of §11.4.144 (device availability-following): same detect-drop → wait/backoff → re-attach/respawn → escalate-if-non-transient shape — devices for §11.4.144, dispatched agents for §11.4.147.

Classification: universal (§11.4.17) — the consuming project supplies its concrete registry path/format, its crash-detection signal source (runtime exit codes / sync-stream error events), its respawn-dispatch mechanism, and its already-defined

backoff budgets per §11.4.35. Composes §11.4.6 (no-guessing — crash ≠ done) / §11.4.58 (PWU = the work unit + worktree isolation) / §11.4.84 (partial-state quiescence check before resume) / §11.4.87 + §11.4.94 + §11.4.97 + §11.4.126 (loop done-condition + zero-idle reclaim) / §11.4.101 (autonomous respawn-vs-restart decision, reversible-with-backup) / §11.4.116 (registry reuses the sync-channel substrate) / §11.4.102 (non-transient terminal error → systematic-debugging before blind respawn) / §11.4.108 + §11.4.125 + §11.4.142 + §11.4.40 (the respawned output still crosses every downstream verification layer) / §9.2 (clean-restart pre-op backup) / §11.4.128 + §11.4.144 (the recording/device-following siblings — same detect→wait→re-attach→escalate shape, agent-side here). Propagation gate `CM-COVENANT-114-147-PROPAGATION` (literal `11.4.147` across the consumer fleet) + recommended gate `CM-CRASHED-AGENT-RESPAWN-TRACKED` (every dispatched agent has a registry entry; every `crashed` entry is respawned-until-`complete`; the loop done-condition reads no non-`complete` entries; partial state preserved + §11.4.84-checked before resume) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; mark a `crashed` entry as the loop's done-condition `complete` without landed evidence, OR drop a dispatched agent without a registry entry → `CM-CRASHED-AGENT-RESPAWN-TRACKED` FAILs; gate-code = separate work item).

(e) API-QUOTA / RATE-LIMIT EXHAUSTION IS A FIRST-CLASS CRASH CLASS (extension, 2026-07-15). Forensic FACT (2026-07-11 + 2026-07-15): background agents were terminated MID-WORK when the driving subscription hit its weekly / session quota — the agents did not fail, they were KILLED by an EXTERNAL quota boundary, and their in-flight work would have been silently lost had a conductor not manually re-dispatched them. Quota exhaustion MUST be handled by the SAME registry + respawn machinery as any other crash, with these additions: (1) the crash-detection layer MUST record the REAL limit signal + its reason-CLASS per §11.4.196(B) (`{session | weekly | subscription}` , derived from the captured 429 signature — never faked, never guessed, §11.4.6); (2) the remedy is NOT "wait for the same alias" — the killed unit MUST be REBOUND to the next OPERATIONAL alias per §11.4.196(A)/(C) (native-first, provider fallback where configured) and RESPAWNED there; when EVERY alias is cooled the unit bounded-PARKS per §11.4.101 carrying its operational-again epoch, and it stays `crashed` / OPEN — never `complete` ; (3) the respawned agent MUST RESUME from the killed agent's captured transcript + on-disk partial state at its EXACT last point (clause (c): preserve → §11.4.84 quiescence check → resume-or-clean-

restart), never restart blind, never silently re-do or drop the work; (4) the §11.4.87 / §11.4.94 / §11.4.97 / §11.4.126 loop done-condition MUST NOT read satisfied while any quota-killed entry is non-`complete` — "the quota ran out" is a PARK, never a DONE. No work may be lost, forgotten, or corrupted by a quota boundary (composes §11.4.196 alias-limit tracking + §11.4.198 always-on multi-alias default — the rebind path is what makes a quota crash recoverable rather than terminal).

Canonical authority: constitution submodule `Constitution.md` §11.4.147. Non-compliance is a release blocker. No escape hatch — no `--skip-agent-registry`, `--crash-equals-done`, `--no-respawn`, `--discard-partial-state`, `--blind-commit-partial`, `--forget-dead-agent`, `--loop-done-with-crashed-entries` flag.

§ . . — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-15): "IMPORTANT: Make sure that recording you perform does record the window containing apps and services, not the whole desktop or monitor screen!" + "NOTE: All old recording files MUST BE removed when new one starts!"

Refines/strengthens the §11.4.2/§11.4.5/§11.4.107/§11.4.153 recording disciplines with two capture-hygiene invariants every feature/QA video MUST satisfy. **(A) Window-scoped capture, NOT whole-screen.** A recording that proves a feature works MUST capture ONLY the window / surface / viewport of the application or service under test — the app's own window (GUI), its terminal pane (CLI/TUI), its browser tab/viewport (web), or its device/emulator/simulator frame (mobile) — NEVER the whole desktop, the whole monitor, or unrelated windows. Whole-desktop capture is forbidden because it (i) leaks unrelated/operator-private content into a committed artefact (a §11.4.10/§11.4.83 hygiene breach), (ii) dilutes the §11.4.107 liveness/freeze/frame-advance oracle with non-feature pixels (a defect can hide in the noise — a §11.4 evidence-integrity weakening), and (iii) makes the §11.4.137-class OCR/ROI content oracle unreliable. The capture mechanism MUST target the window/region by stable identity (window id/title, device serial, browser context, tmux/terminal target) per §11.4.111 — never a fixed full-screen device index that happens to contain the window. Where the platform genuinely cannot capture below whole-screen for the surface under test, that is an honest

§11.4.3 SKIP-with-reason + tracked migration item, NEVER a whole-screen capture passed off as window-scoped. **(B) Fresh-corpus rotation — remove old recordings when a new run starts.** When a new recording run for a given scope begins, the prior run's stale recording files for that scope MUST be removed FIRST, so the live recording corpus always reflects the CURRENT artefact/run and a reader never mistakes a stale video for current evidence (the §11.4.107 not-stale-from-previous principle applied to the corpus, and the §11.4.86 roster-freshness principle applied to video artefacts). Honest boundary (§11.4.6 + §9.2): "remove old" means the agent's OWN prior recordings for the SAME scope/project at the project-declared recording path — NEVER another project's, another scope's, or operator-authored files the agent did not create; when uncertain whether a file is in-scope-stale vs foreign, do NOT delete it — surface it (§11.4.122). Curated evidence already committed under `docs/qa/<run-id>/` (§11.4.83) is the durable record and is NOT deleted by rotation; rotation operates on the raw working recording path (§11.4.128 raw corpus), not the committed evidence trail. Classification: universal (§11.4.17) — the consuming project supplies its concrete window-capture mechanism (per surface class), recording path, and scope-key per §11.4.35. Composes §11.4.2 / §11.4.5 / §11.4.10 / §11.4.83 / §11.4.86 / §11.4.107 / §11.4.111 / §11.4.122 / §11.4.128 / §11.4.137 / §11.4.153 / §9.2 / §11.4.6. Propagation gate `CM-COVENANT-114-154-PROPAGATION` (literal `11.4.154`) + recommended gate `CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING` (every feature/QA recording is window/surface-scoped not whole-screen + a new run removes its own prior in-scope recordings first) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; a whole-screen capture where window-scoped is feasible, or a new run that leaves its own stale prior recordings, → the recording gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.154. Non-compliance is a release blocker. No escape hatch — no `--whole-screen-capture-OK`, `--skip-window-scope`, `--keep-stale-recordings`, `--no-corpus-rotation`, `--full-desktop-recording` flag.

(C) MP4 auto-conversion REQUIRED. Any `.cast` file produced MUST be auto-converted to `.mp4` via `agg` + `ffmpeg` immediately after capture. The `.mp4` is the primary evidence; `.cast` is supplementary only. Conversion command: `agg`

`input.cast output.mp4 --renderer-res 1920x1080` followed by `ffmpeg -i output.mp4 -c:v libx264 -pix_fmt yuv420p -movflags +faststart final.mp4`.

2026 06 15

§ . . — Project-name-prefixed feature/QA recording filenames (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-15): "All recorded videos MUST START with prefix: the PROJECT NAME (ALWAYS USE THE PROJECT NAME). Project name MUST be obtained according to the constitution's own project-name resolution."

Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), and every raw or curated recording artefact under the project-declared recording path (§11.4.35) and the committed `docs/qa/<run-id>/` evidence trail (§11.4.83) — MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS, with no exception. A recording filename that omits the project-name prefix is a §11.4.155 violation: a corpus spanning multiple projects/scopes on one host (a real condition under §11.4.128 always-on recording + §11.4.103 parallel streams) becomes un-greppable and un-attributable, and a reader cannot tell at a glance which project a recording belongs to — the same identify-and-grep failure §11.4.151 forbids on the release-tag axis, applied to the recording-corpus axis.

Prefix resolution order (closed-set, deterministic — §11.4.6 no-guessing; IDENTICAL to §11.4.151's prefix resolution): the project-name prefix MUST be resolved, never guessed, by the constitution's own project-name resolution: (1) `HELIX_RELEASE_PREFIX` from the project's `.env` — authoritative when set; `.env` is git-ignored per §11.4.30 and the variable is documented in the tracked `.env.example` (a §11.4.77 re-obtain mechanism, never committed); (2) fallback = the lowercased snake_case form of the project root directory name (no spaces) per §11.4.29 — used whenever the env var is unset/empty, so a prefix is ALWAYS resolvable from the checkout with zero operator input. The SAME resolved prefix is used for EVERY recording the project produces in a given checkout, so a single `ls`

`'<PREFIX>---'*` (or `find . -name '<PREFIX>---*'`) enumerates the whole recording corpus for that project. Canonical filename form: `<PREFIX>---<feature-or-scope>---<run-id>.<ext>` (the `---` triple-hyphen separator keeps the prefix unambiguously delimited from a feature/scope name that may itself contain hyphens). The prefix MUST be the SAME value §11.4.151 resolves for release tags in the same checkout — a recording prefix and a release-tag prefix diverging in one checkout is itself a §11.4.155 violation (one project, one resolved name).

Honest boundary (§11.4.6): the project-name prefix guarantees a recording is attributable + greppable to its project, NOT that the recording's CONTENT is valid — content validity still rests on the §11.4.107 liveness battery, the §11.4.137 content-correctness oracle, and the §11.4.153 video-analysis remediation loop; the prefix is a naming/attribution discipline that composes with, never replaces, those evidence layers. The prefix also does NOT relax §11.4.154's window-scoped-capture + fresh-corpus-rotation invariants (rotation removes the agent's OWN prior in-scope `<PREFIX>---*` recordings first; a foreign-prefix or operator-authored file is surfaced, never deleted, per §11.4.122 + §9.2).

Classification: universal (§11.4.17) — a platform-neutral recording-attribution discipline reusable by ANY project that produces recordings; the consuming project supplies its concrete prefix value + the `HELIX_RELEASE_PREFIX` env var + its recording path per §11.4.35. Composes §11.4.151 (the SAME prefix-resolution order + the same identify-and-grep purpose, applied to recordings instead of release tags) / §11.4.128 (every always-on device recording filename carries the prefix) / §11.4.153 (the per-feature real-use video's confirmation path is prefixed) / §11.4.154 (window-scoped + fresh-corpus rotation operate on prefixed filenames) / §11.4.111 (the prefix is a stable-identity name, not an enumeration index) / §11.4.83 (committed `docs/qa/<run-id>/` recording evidence carries the prefix) plus §11.4.6 / §11.4.29 / §11.4.30 / §11.4.35 / §11.4.77 / §11.4.86. Propagation gate `CM-COVENANT-114-155-PROPAGATION` (literal `11.4.155` across the consumer fleet) + recommended gate `CM-RECORDING-PROJECT-NAME-PREFIX` (every recording filename at the project's recording path + every committed `docs/qa/<run-id>/` recording artefact starts with the resolved `<PREFIX>---` prefix, identical to the §11.4.151-resolved prefix for the checkout) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; write a recording with no project-name prefix, or a prefix differing from the §11.4.151-resolved value → `CM-RECORDING-PROJECT-NAME-PREFIX` FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.155. Non-compliance is a release blocker. No escape hatch — no `--no-recording-prefix`, `--recording-without-project-name`, `--unprefixed-recording`, `--prefix-optional-for-recording`, `--differing-recording-prefix` flag.

§11.4.174 — Shared-host process-ownership verification mandate (User mandate, 2026-06-29). Verbatim operator mandate: "Make sure that processes we are checking are always OURS! Add this as mandatory constitution rule / constraint, since on every host there may be other work being done!" On ANY host where concurrent work from other projects / users / agents may run (the normal case — a shared developer workstation), EVERY inspection of, conclusion about, or action on a process, build, daemon, port, lock-file, or system-state signal MUST FIRST positively VERIFY the target is OURS (this project's) before drawing a diagnosis or acting on it. A diagnosis based on a process that is actually another project's work is a §11.4.6 guessing-class error; KILLING / signalling / restarting / altering a process that is NOT positively ours is a §12 + §11.4.133-class safety violation (operator-workload destruction) — forbidden. Forensic anchor (FACT, 2026-06-29): during an ATMOSphere build-stall diagnosis, a 200%-CPU `java` + `gradlew` on the shared host were nearly mis-attributed to our SmartTube build; they were in fact a separate `catalogizer` project's gradle test (`cwd=Projects/catalogizer`), and a naive `pgrep java` / `gradle` would have matched them as "ours." The mandate (ALL hold): (a) **positive-ownership discriminator REQUIRED** — attribute a process to us ONLY via a strong signal: its `cwd` resolves inside THIS project's tree, its `argv` names this project's scripts / artifacts / repo path, a PID we ourselves launched and recorded (the §11.4.147 launched-PID registry), or a lock / port / socket path under our tree. A loose name match (`pgrep java` / `gradle` / `node` / `python` / `podman` / `zig`) is NEVER sufficient — it matches every project on the host. (b) **NEVER kill / signal / restart a process not positively ours** — other projects' gradle / java / kotlin daemons / podman pods / build processes are strictly off-limits (operator-workload-safety); the §12.8 remediation `pkill -f GradleDaemon` MUST be scoped to OUR daemons (cwd / argv under our tree), never a blanket host-wide kill. (c) **contention → surface, don't break** — when our work waits on host contention caused by another project's process (e.g. our §12.8 build gate sleeping because another project's gradle daemon is alive), surface it per §11.4.66 / §11.4.101 (the safe blocked-decision) and keep progressing non-blocked work; do NOT kill the other project's process to unblock ours. (d) **filters exclude self + others** — every `pgrep` / `ps` filter MUST

exclude the checking command's OWN argv (the self-match that produced the false "still running" restyle watcher bug, 2026-06-29) AND other-project matches; prefer matching on our absolute repo path. (e) **applies to all state, not just processes** — ports, lock files, gradle/maven caches, podman pods, tmux sessions, temp files on a shared host all belong to potentially-other work; verify ownership before reading-as-ours or mutating. Honest boundary (§11.4.6): ownership-verified inspection prevents mis-attribution + cross-project damage — it does not itself prove our process is healthy (that still needs our own artifacts: our build log, our output files, our launched-PID). Composes §11.4.6 (no-guessing) / §11.4.101 (autonomous safe blocked-decision — block-don't-break) / §11.4.66 (surface contention) / §11.4.147 (launched-PID registry as the ownership source) / §12 + §12.8 (host-safety, now scoped to our daemons) / §11.4.133 (don't break other workloads). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-174-PROPAGATION` (literal `11.4.174`) + recommended gate `CM-PROCESS-OWNERSHIP-VERIFIED` (process/daemon-killing or build-state-diagnosing scripts attribute by cwd/argv/launched-PID, never a bare name match; pgrep filters exclude self) + paired §1.1 mutation (replace an ownership-verified attribution with a bare `pgrep <name>` → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). Non-compliance is a release blocker. No escape hatch — no `--assume-process-ours`, `--loose-pgrep-ok`, `--kill-any-matching-process`, `--skip-ownership-check` flag.

§ . . — Scheduler-quota burst-throttling telemetry + interactive-scope isolation from bursty fleets: progressive interactive degradation is diagnosed from throttling accounting over time, never from averages, and an interactive-latency-sensitive process never shares a fixed-quota no-burst scope with a bursty fleet (research-derived, - -)

Forensic anchor (genericised, 2026-07-22 — two independent instances in one day). (a) Progressive interactive sluggishness — worsening with the number of live subagents — was root-caused to a scheduler CPU quota (a cgroup `cpu.max` limit) with NO burst allowance on a session scope hosting BOTH the operator's interactive terminal-multiplexer server AND a spawning agent fleet. Measured live

on a 64-CPU host: the scope's quota was 9.6 CPU per 100 ms enforcement period; average demand was 4.35 CPU — comfortably UNDER quota — yet 51 of 301 periods (16.9%) were throttled during active phases, ~6.4 runqueue-seconds of forced idle per wall-second at peak, 0% throttled during quiet phases (episodic, fleet-phase-correlated), and the lifetime ratio (10.9%) vs the live ratio (16.9%) showed it WORSENING with session age. The terminal server was co-resident in the throttled scope: it froze for ≥100 ms per throttled period, and consecutive throttled periods compounded into multi-second interactive stalls. Every conventional instrument read HEALTHY — CPU average under quota, load average unremarkable, memory fine, thread counts fine — the degradation was INVISIBLE to averages and lived entirely in the throttling accounting (`cpu.stat` `nr_throttled` / `throttled_usec` deltas). (b) A sibling session launcher shipped a FIXED default quota (2 CPUs) on the same 64-core host — 18.8% of periods throttled and 1757 s of cumulative forced idle — fixed by computing the default from real detected host capacity (a per-core fraction with a floor and a cap), proven by a four-quadrant polarity test reading the LIVE cgroup back.

Two operative halves, BOTH mandatory:

(A) DIAGNOSTIC — throttling accounting over time under load, never averages. Any investigation of progressive / bursty / load-correlated interactive degradation (sluggish TUI or terminal, editor lag, stalls that worsen with parallel-agent or worker count) MUST sample the scheduling scope's THROTTLING ACCOUNTING — on Linux cgroup v2, `cpu.stat` `nr_throttled` + `throttled_usec` as DELTAS over a measured window under load, expressed as a fraction of elapsed enforcement periods, WITH a quiet-phase control sample (the §11.4.201(7)(b) needle in time: a mechanism that fires 17% under load and 0% quiet is proven load-coupled, not ambient) — BEFORE any verdict on the cause is minted. A HEALTHY AVERAGE WITH A HIGH THROTTLE RATIO IS THE TRAP: quota enforcement is per-period (canonically 100 ms), so a workload can sit far below its quota ON AVERAGE while a large fraction of individual periods hit the ceiling and stall every co-resident process; CPU averages, load average, memory, and thread counts are all structurally BLIND to it — their quiet "healthy" readings are §11.4.201(6) FALSE-NULLS, and "the averages look healthy, so the host is fine" without throttle-accounting deltas is a §11.4.6 guess. The same discipline applies to EVERY quota'd scheduler (container CPU limits, VM caps, hypervisor

shares / steal time): the throttle/steal accounting IS the authoritative source (§11.4.201); the utilisation average is a proxy that the real condition does not have to satisfy.

(B) DESIGN — an interactive-latency-sensitive process NEVER shares a fixed-quota, no-burst scope with a bursty fleet. An interactive-latency-sensitive process (terminal-multiplexer server, TUI, editor, operator shell) MUST NOT be co-resident in a fixed-quota scheduling scope that has NO burst allowance together with a bursty multi-process fleet (spawning subagents, build workers, test runners): the fleet's demand spikes consume whole enforcement periods and the interactive process freezes with them — in bursts, progressively as the fleet grows (the §11.4.103/§11.4.183 parallel-stream mechanics make the fleet's demand spiky BY DESIGN, so the collision is structural, not incidental). The consuming project MUST (1) grant the shared scope a burst allowance sized for the fleet's measured spike profile, OR (2) isolate the interactive process in its OWN scope (or leave it unquoted where host policy allows), OR both. Quota values MUST be HOST-ADAPTIVE — computed from real detected capacity with a documented floor and cap (§12.11's dynamic-detection discipline; §11.4.6 never hardcode), never a fixed literal that silently strangles a many-core host (the forensic clause-(b) instance); and a quota change is verified by READING THE LIVE scope's values BACK (§11.4.108 runtime signature — the configured value is not the effective value until read back). Composes §12.6 (memory ceiling) + §12.12 (thread/RLIMIT_NPROC headroom) as the THIRD orthogonal host-exhaustion axis: abundant RAM and abundant thread headroom do NOT imply an un-throttled scheduler — each axis has its own authoritative instrument, and none substitutes for another.

Honest boundary (§11.4.6): throttle telemetry proves WHERE enforcement periods were lost — it does NOT prove throttling is the ONLY degrader (network RTT floors, in-process GC/event-loop load, and sink-side effects remain separately measurable; the founding investigation ranked them independently and REFUTED five candidate mechanisms with the same instrumentation rigor); and clause (B) bounds PLACEMENT + SIZING — it does not exempt the fleet from the §12.6/§12.8/§12.12 host-safety ceilings, and it never licenses removing a quota that host policy requires (widen/isolate/burst within policy, per §12).

Classification: universal (§11.4.17) — scheduler-quota throttling is a platform-general mechanism (cgroup v1/v2, container runtimes, VM hypervisors); the consuming project supplies its concrete scope layout, quota/burst values, sampling

windows, and instruments per §11.4.35. Composes §11.4.6 / §11.4.24 (build-resource stats — the sampler seam this telemetry extends) / §11.4.102 (systematic-debugging — the throttle sample is Phase-1 fact-gathering, and the forensic investigation reached it only by refusing to stop at healthy averages) / §11.4.108 (read the live scope back) / §11.4.128(1) (observer-effect budget — the sampler itself stays lightweight) / §11.4.201 (authoritative-source measurement; the averages-only verdict is a false-null) / §12.6 / §12.8 / §12.11 / §12.12. Propagation gate `CM-COVENANT-114-225-PROPAGATION` (literal `11.4.225`) + recommended gates `CM-QUOTA-THROTTLE-TELEMETRY` (every progressive/bursty-degradation investigation's verdict cites throttle-accounting DELTAS sampled over a measured window under load + a quiet-phase control; an averages-only verdict on a progressive-degradation issue → FAIL) and `CM-INTERACTIVE-SCOPE-NOT-QUOTA-STARVED` (no interactive-latency-sensitive process is co-resident in a fixed-quota no-burst scope with a bursty fleet; quota defaults are host-adaptive with a live read-back, never fixed literals) + paired §1.1 mutations (record a progressive-degradation verdict citing only averages → the telemetry gate FAILs; pin a fixed low quota literal as the default on a detected many-core host, or co-locate the terminal server with the fleet in a no-burst quota scope → the isolation gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.225.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--averages-prove-healthy`, `--skip-throttle-telemetry`, `--fixed-quota-default-OK`, `--interactive-in-fleet-scope-OK`, `--no-burst-needed`, `--skip-quiet-phase-control` flag exists.

11 4 254

2026 08 15

§ . . — Boot-time invariant assertion + capability matrix (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.254. Every service / process / long-running executable MUST assert BOOT-TIME INVARIANTS at startup — required env vars present + valid; required config files parseable + declared schema version matches; required deps reachable; required credentials present + not-expired; required schema migrations applied; required feature-flag states declared; required host resources present (disk / memory / thread limits per §12.12 / port availability) — and FAIL FAST + LOUD if any violated, BEFORE accepting request / performing

work. Extraction line 6488 [offset un-resolvable per the §11.4.239 CITATION NOTE; **MODULE ADDED 2026-08-20** — resolves to **module 35** taxonomy row 6 "Config-present-but-unwired", verified locators `module_35:113` (row header), `module_35:116` (the case), `module_35:118` (the boot-time `cipher_version` remedy)] (Catalogizer SQLCipher case: `catalog.db` 's first 16 bytes are ASCII `SQLite format 3` — plaintext despite "encrypted at rest"; fix = boot-time-check-that-fails-closed verifying `cipher_version` at startup) + line 6489 [**MODULE ADDED 2026-08-20** — resolves to **module 35** taxonomy row 7 "Stubbed-core-behind-claim", verified locators `module_35:120` (row header), `module_35:123` (the Catalogizer nil,nil-providers case), `module_35:125` (the `IsImplemented()` capability-matrix remedy)] (stubbed-core-behind-claim + explicit capability matrix with every adapter reports `IsImplemented()` /attempted-vs-not + challenge that fails against no-op stub). Prevents "service silently starts with broken invariant + serves requests that fail deep in request path hours after boot." INVARIANT CLASSES: (i) environment, (ii) configuration, (iii) dependencies, (iv) credentials, (v) schema, (vi) capabilities, (vii) host. FAIL-FAST-LOUD: (1) EMIT structured error to stderr + captured log naming specific failed invariant; (2) EXIT non-zero (traditionally 78 EX_CONFIG for config, consumer supplies per §11.4.35); (3) DO NOT accept requests / begin main event loop / bind listening port. Service booting partially + serving requests-that-will-fail is worse than refusing to start — partial-boot masquerades healthy long enough for load balancer to route traffic. Dual to §11.4.252 fail-closed on dangerous combinations. CAPABILITY MATRIX: consumer's `docs/services/CAPABILITY_MATRIX.md` (per §11.4.35) declares per service (i) boot-time invariants, (ii) runtime capabilities OFFERED, (iii) runtime capabilities REQUIRED from other services. Regenerated / verified on every commit per §11.4.106 docs-chain + §11.4.86 fingerprint. Undeclared endpoints emitted = violation; silent upstream deps not declared = violation (invisible to §11.4.244 contract testing). Composes §11.4.5 / §11.4.6 / §11.4.35 / §11.4.86 / §11.4.106 / §11.4.108 Layer 3 (boot IS runtime-on-clean-target check) / §11.4.128 / §11.4.244 (contract tests consume matrix) / §11.4.252 / §12.12 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-254-PROPAGATION` (literal `11.4.254`) + recommended gates `CM-BOOT-TIME-INVARIANT-ASSERTED` + `CM-CAPABILITY-MATRIX-PRESENT-AND-IN-SYNC` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.254. Non-compliance is a release blocker. No escape hatch — no `--skip-boot-time-assertion`, `--boot-partial-OK`, `--default-missing-env-silently`, `--capability-matrix-optional`, `--service-with-undeclared-dependency-OK` flag.

---

§ . . —Process-group signal-safety mandate: NEVER signal pgid ≤ , NEVER trust a mock-derived pid/pgid — validate as int > before every `killpg` / `kill(-pid, sig)` / `pkill` / `killall` call (BOB- forensic anchor, - -)

Forensic anchor — verbatim operator mandate (2026-08-19):

> "CRITICAL: Fix MUST BE systhematic so any other projects do not have same issue!"

Forensic case study (BOB-126, boba project, 2026-08-19). SEVEN forced-logout incidents on the operator's workstation over ~48 hours (BOB-116/120/123/124/125/126) — the operator's entire graphical session, tmux, SSH, browsers, and Claude Code process were repeatedly SIGKILLed. Kernel audit rules installed 2026-08-19 15:56 finally captured the initiator on incident #7:

```
audit[399861]: SYSCALL syscall=62 a0=ffffffff a1=9
  pid=399861 auid=1000 comm="pytest" exe="/usr/bin/python3.14"
  key="sigkill_investigation"
```

A `pytest` process called `kill(-1, SIGKILL)`. Root cause: a Python test created an `AsyncMock()` subprocess object without explicitly setting `mock.pid` as an int. The production code path called `os.killpg(os.getpgid(proc.pid), signal.SIGKILL)`. Python's `MagicMock.__int__` defaults to **1**, so:

1. `proc.pid` → `MagicMock`

2. Python coerces to `int` for `os.getpgid` → **`int(MagicMock()) == 1`** (documented default)
3. `os.getpgid(1)` returns **1** (init's process group)
4. `os.killpg(1, SIGKILL)` → glibc → **`kill(-1, SIGKILL)`**
5. `kill(-1, sig)` semantics = "signal every process the caller may signal, except pid 1 and self" → under UID 1000 that means the entire user session (systemd `user@1000` manager, gnome-shell, tmux, ssh, browsers, Claude Code)

The `contextlib.suppress(Exception)` around the call swallowed nothing because the syscall SUCCEEDED before any exception path. The defect existed for ~4 months (since 2026-04-24) before audit rules exposed it.

The mandate (ALL hold):

(A) NEVER signal pgid ≤ 1. No project, no language binding, no code path may call `os.killpg(pgid, sig)` / `kill(-pid, sig)` / `killpg(2)` (POSIX) / `pkill -<pgid>` / `subprocess.killpg` / `Process.killpg` where `pgid` (or `pid` in the negative-pid form of `kill(2)`) is `≤ 1`. On Linux, `killpg(1, sig)` and `kill(-1, sig)` are equivalent and BOTH signal every process the caller has permission to signal — under a non-root UID, that includes the user's session manager, GUI, shell, and every daemon in the same UID scope. This is the disaster syscall. Guarding with a positive integer test is trivial; failing to guard is catastrophic. On other Unixes (BSD family, macOS) the semantics of `kill(-1, sig)` is broadly the same "signal all processes the caller may signal"; the mandate is portable.

(B) VALIDATE pid + pgid as `int > 1` BEFORE every process-group signal call. The universal invariant:

```
IF calling killpg(pgid, sig) OR kill(-pid, sig) OR any process-group
signal:
  ASSERT isinstance(pgid, int)         # never a mock, never a string,
never None
  ASSERT pgid > 1                      # never init's pgid, never 0 (self-
group), never negative
  (same for pid when the negative-pid kill(2) form is used)
```

Language-specific enforcement:

- **Python**: guard with `if isinstance(pid, int) and pid > 1: pgid = os.getpgid(pid); if isinstance(pgid, int) and pgid > 1: os.killpg(pgid, sig)`.
- **Go**: `syscall.Kill(-pgid, sig)` — guard `pgid > 1` before the call; test doubles must return `int(pgid) > 1`.
- **Rust**: `nix::sys::signal::killpg` — accept only `Pid::from_raw(n)` with `n > 1`, and reject the sentinel meaning "current process group".
- **Bash / shell**: `kill -<pgid> <sig>` / `pkill -g <pgid>` — validate `[[ "$pgid" -gt 1 ]]` before the call.
- **C**: `killpg(pgid, sig)` and `kill(-pid, sig)` — same integer guard.

(C) TESTS mocking subprocess/proc objects MUST explicitly set `mock.pid` (or equivalent) as int. A subprocess mock with an unset `pid` field is BOB-126-shaped by construction: the language's mock library (Python's `MagicMock`, Go testify's `Mock`, JS Jest's `jest.fn()`, etc.) provides a default coercion that can silently become a low integer. Every test that creates a fake subprocess object MUST:

1. Set `mock.pid = <large int, e.g. 12345>` explicitly — never rely on the mock library's default.
2. Patch the process-group signaling call itself (e.g. `patch.object(module.os, 'killpg')`) as belt-and-suspenders so a future regression of the production code's guard cannot re-open the disaster.
3. Assert the mocked `killpg` was called with `pgid > 1` (paired §1.1 mutation catches a regression that lets pgid=1 through).

(D) DEFENSE-IN-DEPTH (§11.4.107(10) analyzer discipline applied). The guard MUST be present in BOTH the production code AND the test. Removing either layer alone must break at least one test. This is the four-layer coverage per §11.4.4(b) applied to the signal-safety class:

- Layer 1 (pre-build gate): grep pattern for `killpg\(.*\)` / `kill\(-.*\)` calls that are NOT preceded by the guard — surface as review findings.
- Layer 2 (post-build gate): CodeGraph query for callers of `os.killpg` / `syscall.Kill(-*)` / equivalent, cross-referenced against guard presence.

- Layer 3 (runtime): kernel audit rules on SIGKILL syscalls with `a0=0xffffffff` (kill(-1)) or `a0=0x00000001` in the pidfd_send_signal case — a live host-safety canary.
- Layer 4 (paired §1.1 mutation): a regression test that DELIBERATELY invokes the killpg code path with a mock pid AND asserts `killpg` was not called with `pgid ≤ 1`. Reverts of the pid guard MUST make this test fail.

(E) HONEST BOUNDARY (§11.4.6). §11.4.263 hardens the process-group signal APIs specifically. It does NOT prohibit legitimate use of `killpg` for well-scoped process groups (`start_new_session=True` subprocesses where the child owns its group). It does NOT extend to signals other than SIGKILL where the blast radius is small (SIGCHLD, SIGUSR1, etc. — though the same guard is cheap and recommended). It does NOT prevent kernel OOM-killer or systemd-oomd from killing the user (those are separate host-safety concerns — §12.6 memory ceiling + §11.4.225 quota telemetry apply).

(F) STANDING DEFAULT. Every project inheriting this constitution treats §11.4.263 as always-on from the first prompt. Every code review (§11.4.125/§11.4.142/§11.4.194) MUST scan for process-group signal calls and verify the guard. Every subagent dispatched to write subprocess-cleanup code MUST include the guard in its output. Every test-writing subagent MUST include the `mock.pid = <int>` discipline.

Classification: universal (§11.4.17). Composes §11.4.1 (FAIL-bluff — swallowed exception around a successful catastrophic syscall is a bluff at the error-handling layer) / §11.4.4(b) (four-layer coverage) / §11.4.5 / §11.4.6 / §11.4.85 (chaos-test host safety) / §11.4.107(10) (self-validated analyzer) / §11.4.115 (RED-first with the actual defect condition) / §11.4.126 (standing default) / §11.4.201 (guard asserts real condition — pgid > 1) / §11.4.225 (host-safety orthogonal axis) / §11.4.226 (real captured evidence). Propagation gate `CM-COVENANT-114-263-PROPAGATION` (literal `11.4.263`) + recommended gates `CM-KILLPG-PGID-GUARD` (production code — every killpg/kill(-pid) call site has the `pid>1 && pgid>1` guard) + `CM-TEST-MOCK-PID-EXPLICIT-INT` (test code — every subprocess mock has `mock.pid` explicitly set as int + killpg patched) + paired §1.1 mutations (remove the guard from production → the regression test fails; unset `mock.pid` in the test → assertion catches pgid<=1 attempt).

Canonical authority: constitution submodule `Constitution.md` §11.4.263. Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-killpg-guard`, `--mock-pid-optional`, `--killpg-1-is-ok`, `--kill-minus-1-permitted`, `--suppress-catches-syscall`, `--test-may-not-explicit-pid` flag exists.

---

§ . Forbidden operations — directly OR indirectly

1. **Suspending the host**: `systemctl suspend`, `pm-suspend`, `loginctl suspend`, DBus `org.freedesktop.login1.Suspend`, GNOME idle-suspend, lid-close handler.
2. **Hibernating / hybrid-sleeping**: any `Hibernate` / `HybridSleep` / `SuspendThenHibernate` method.
3. **Logging out the user**: `loginctl terminate-session`, `pkill -u <user>`, `systemctl --user --kill`, anything that signals `user@<uid>.service`.
4. **Unbounded-memory operations** inside `user@<uid>.service` cgroup. Any single command expected to exceed ~4 GiB RSS MUST be wrapped in a bounded execution scope (e.g. `bounded_run` from a project helper library that wraps `systemd-run --user --scope -p MemoryMax=...`).
5. **Programmatic rfkill toggles, lid-switch handlers, or power-button handlers** — these cascade into idle-actions.
6. **Disabling systemd-logind, GDM, or session managers** "to make things faster" — even temporary stops leave the system unable to recover.

§ . Required safeguards

Every script in this project that performs heavy work (build, transcription, model inference, large compression, multi-GB git operations) MUST:

1. Source the project's host-safety helper library at the top.
2. Call its pre-flight check and **abort if it fails**.
3. Wrap any subprocess expected to exceed ~4 GiB RSS in a bounded execution scope so the kernel OOM killer is contained to that scope and cannot escalate to user.slice.

4. Cap parallelism (`-j`) to fit available RAM (estimate per-job-peak-RSS and divide into the budget).

§ . Container hygiene

Containers (Docker / Podman) the project owns or relies on MUST:

1. Declare an explicit memory limit (`mem_limit` / `--memory` / `MemoryMax`).
2. Set `OOMPolicy=stop` in their systemd unit to avoid retry loops.
3. Use exponential-backoff restart policies, never immediate retry.
4. Be clean-slate destroyed and rebuilt after any host crash or session loss so stale lock files don't keep producing failures.

§ . Memory-Budget Ceiling — % MAXIMUM

Forensic anchor — direct user mandate:

> "First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

The mandate. Project procedures MUST NOT use more than **60% of total system RAM**. The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

The protections:

1. `HOST_SAFETY_MAX_MEM_PCT` defaults to 60.
2. `HOST_SAFETY_BUDGET_GB` is computed at source-time from `MemTotal × MAX_PCT/100` , in GiB.
3. Bounded execution scopes clamp `MemoryMax` down to the budget if the caller asks for more.
4. The build script's parallelism is computed as `min(nproc, floor(budget_gb / per_job_peak_rss_gb))` .

5. Heavy work MUST be wrapped in a bounded execution scope so the kernel OOM-kills only the scope — `user@<uid>.service` stays alive.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits.

§ . Continuation document — sacred invariant

Forensic anchor — direct user mandate:

> "during any work we perfrom, during Phases implementation, debugging and fixing, during ANY effort we have the Continuation document MUST BE maintained and it MUST NOT BE out of sync with current work we are doing! If for any reson we stop our work, we MUST BE able to continue any time, with current work, exactly where we have left of and from any CLI agent or any LLM model we chose! Nothing can be broken or faulty in maintained Continuation document!"

The mandate. A single, canonical, machine-readable handoff document — `docs/CONTINUATION.md` — must always reflect the live state of the project. Any agent (human, Claude Code, Cursor, Aider, Codex, Gemini CLI, any future LLM) must be able to resume work **exactly where the previous session left off** by reading this single file.

Mandatory protections:

1. **`docs/CONTINUATION.md` MUST exist** at the project root. Its absence is a release blocker.
2. **Every non-trivial state change** MUST update this document in the same commit as the work itself.
3. **Top-of-file timestamp** is updated on every edit. Stale timestamps trigger gate failure.
4. **Section §3 "Active work"** lists every IN PROGRESS / BLOCKED item with enough detail that any agent can resume without conversation context — concrete commands, file paths, monitor IDs.

5. **Section §0 "How to use this document"** contains the verbatim resumption prompt — a single block any operator can paste into any CLI agent.
6. **Document MUST be self-contained.** No hyperlinks to ephemeral external systems as the only source of truth.

No escape hatch. §12.10 has NO operator-facing override flag for the existence requirement. The discipline exists for the operator's own protection.

§ . — Maximal dynamic resource utilization for containerized builds (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-07-03):

> "Use full workstation system capacity — all resources to build and test! Give maximal number of cores and memory! Use dynamic capabilities! No limiting to -j2 anymore! ... MUST NOT break the system, create bottlenecks, or get stuck! ... MUST BE now part of constitution Submodule! On each System we use, aim for maximal use of available resources for maximal efficiency!"

The reconciliation (the load-bearing logic). The existing memory-and-parallelism caps were born from a specific, forensically-captured failure mode: **heavy build work running inside the user login session's cgroup (`user@<uid>.service` / `user.slice`) overran memory and the kernel OOM-killer escalated to the session itself, SIGKILLing the operator's desktop, terminals, and the agent** (the §12.6 forensic anchor: "three consecutive session-loss SIGKILLs" during a bare-metal `m -j5`; consuming-project instantiations extend this — a bare-metal AOSP `m -j` was hard-capped at -j2 because `soong_build` alone holds a large constant RSS inside user.slice). Those caps — **§12.6's 60%-of-total-RAM ceiling** and the consuming project's bare-metal `-j` cap — are CORRECT and REMAIN in force for **any heavy work that executes inside `user@<uid>.service` / `user.slice`**, because such work shares the session's OOM fate.

A build that runs in **its OWN container cgroup with its OWN `MemoryMax`** (the rootless-Podman containerized build path — generically "the sanctioned containerized build") is a **fundamentally different accounting domain**: it is NOT a child of `user.slice`, so the kernel OOM-killer and any userspace OOM watchdog

(systemd-oomd's `user.slice` PSI watch) act on the **container's own cgroup FIRST** — the container OOM-kills ITSELF, it does not escalate to and cannot SIGKILL the operator session. Therefore a high memory / CPU / `-j` envelope in the containerized path is **host-SAFE by construction** — the exact hazard the §12.6 ceiling guards against does not reach the session across the cgroup boundary.

The mandate. For the **containerized build path only**, the project MUST use the **maximal SAFE fraction** of the host's real capacity, computed **dynamically per host** — NOT a fixed low `-j` count, NOT clamped to the §12.6 60% figure (which governs user.slice-resident work, a different domain). Concretely (ALL hold): (1) **Auto-detect host capacity, never hardcode** — read this host's real `nproc`, `MemTotal`, `SwapTotal`, and the hard process/thread rlimit (`RLIMIT_NPROC` / `ulimit -Hu`) at launch; every number driving the envelope MUST be measured on the running host, never assumed (§11.4.6 no-guessing); a launcher that prints a build envelope it did not measure is a §11.4.6 violation. (2) **Reserve explicit host headroom (cores AND RAM)** — the maximal-safe fraction is "all capacity MINUS a reserved slice kept free for the operator's live session + other workloads"; the headroom MUST be explicit and captured in the printed envelope; this preserves the SPIRIT of §12.6 (the operator's session keeps functioning) via a reserved-headroom mechanism, WITHOUT importing §12.6's 60% number — the container's cgroup, not a user.slice budget line, bounds it. (3) **`-j` scales nproc-awarely, not just memory-awarely** — the parallel-job count MUST respect BOTH the memory governor (`floor((container_mem_gb − build_constant_overhead_gb) / per_job_peak_rss_gb)`) AND the process/thread rlimit, because each `-j` unit spawns compiler/linker worker THREADS that count against `RLIMIT_NPROC` and exceeding it causes `EAGAIN` "fork: retry" storms that stall/wedge the build (the "never get stuck" clause); where the login's hard `nproc` is too low for the full memory-and-CPU-derived `-j`, the launcher MUST either (a) run at the proven-safe lower `-j` under the rlimit, OR (b) on a privileged path raise the rlimit (e.g. `prlimit`) and drop back to the unprivileged uid (e.g. `setpriv`) before launching so the container keeps its rootless identity; the launcher MUST print which mode it chose (§11.4.6 — the envelope is captured evidence). (4) **The caps REMAIN for the bare-metal / user.slice path** — §12.6's 60% ceiling and the consuming project's bare-metal `-j` cap are NOT relaxed for any work inside `user@<uid>.service` ; §12.11 grants the maximal envelope ONLY across the container-cgroup boundary; a bare-metal heavy build is still forbidden from claiming

the §12.11 envelope (that would reproduce the original session-loss incident); §12.11 does NOT weaken §12.6, it SCOPES it. (5) **Never break, never bottleneck, never get stuck** — the envelope MUST leave the reserved host headroom genuinely free (verifiable post-launch: session memory-available stays above the reserved floor), MUST NOT drive swap-thrash inside the container (bound `MemorySwapMax` so the container hits its own OOM before pathological swap), and MUST NOT exceed the process rlimit (per clause 3); "maximal" means maximal-SAFE, not maximal-reckless — an envelope that wedges the host is a §12.11 violation, not compliance. (6) **Evidence-based tuning, no guessing (§11.4.6 + §11.4.24)** — the chosen envelope is driven by captured host metrics + the build's own resource profile (§11.4.24 build-resource stats); "use a bigger `-j`" without captured headroom evidence proving the host can absorb it is a §11.4.6 + §12.11(5) violation; when in doubt, the reserved-headroom floor wins.

Reference implementation (consuming project). A thin DYNAMIC wrapper over the sanctioned containerized build (ATMOSphere: `scripts/build_maxres.sh` over `scripts/build_containerized.sh`) reserves a fixed core count + a RAM floor (e.g. 72% of MemTotal to the container, clamped so a fixed-GiB slice stays free for the session), computes `JOBS_MEM = floor((container_mem_gb − constant_overhead_gb) / per_job_peak_rss_gb)`, and governs `-j` against the real hard `nproc`: unprivileged → the proven-no-`EAGAIN` job count; privileged (root re-exec raises `RLIMIT_NPROC` via `prlimit` then `setpriv`-drops back to the unprivileged uid) → full `-j = min(cpus, JOBS_MEM)`; `--dry-run` prints the full envelope and launches nothing. This is the §11.4.35 project instantiation; the mechanism above is universal.

Composes with §12.1 / §12.2 / §12.3 (host-session safety — §12.11 is a strengthening within, not a relaxation of, host safety) / **§12.6** (the 60% ceiling — §12.11 SCOPES it to user.slice-resident work, does NOT weaken it) / §12.10 (the launcher's envelope is a live-state anchor for the resumption file) / §11.4.6 (no-guessing — every envelope number measured) / §11.4.24 (build-resource stats feed the tuning) / §11.4.133 (target/hardware safety — the reserved-headroom + rlimit governors are the host-safety equivalent) / §9.2 (the containerized path is the non-destructive default). The consuming project supplies its containerized-build path, cgroup mechanism, and per-host detection per §11.4.35.

Classification: universal (§11.4.17) — the reconciliation logic (caps bind user.slice-resident work; the containerized cgroup is separately accounted so it may claim the maximal safe fraction) is platform-agnostic and true on every host. Propagation gate `CM-COVENANT-12-11-PROPAGATION` (literal anchor `12.11` present across the consumer fleet's CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md) + recommended gate `CM-MAXRES-DYNAMIC-BUILD` (the containerized-build launcher (a) auto-detects `nproc` / `MemTotal` / `RLIMIT_NPROC`, (b) reserves explicit host headroom, (c) governs `-j` against BOTH the memory model AND the process rlimit, (d) prints the chosen envelope, (e) does NOT hardcode a fixed low `-j` for the containerized path AND does NOT relax §12.6 for any user.slice path) + paired §1.1 meta-test mutations (strip the `12.11` literal → propagation gate FAILs; replace the dynamic detection with a hardcoded `-j2` in the containerized launcher OR strip the rlimit-aware `-j` governor → `CM-MAXRES-DYNAMIC-BUILD` FAILs; gate-code = a separate tracked work item).

Canonical authority: constitution submodule `Constitution.md` §12.11. Non-compliance is a release blocker regardless of context. No escape hatch — no `--fixed-jobs`, `--no-dynamic-detection`, `--ignore-host-capacity`, `--relax-user-slice-cap`, `--bare-metal-maximal` flag exists.

2026-07-07

§12.11 — Process/thread-limit (RLIMIT_NPROC) awareness for parallel subagent/multi-process work (User mandate, 2026-07-07)

Forensic anchor — verbatim user mandate (2026-07-07):

> "Add these ulimit details and instructions to the constitution so they are mandatory for this and all projects!"

The limit. Heavy parallel subagent / multi-process work is bounded by the OS per-user process/thread limit `ulimit -u` (`RLIMIT_NPROC`), which on Linux counts ALL threads AND processes owned by the real UID — NOT a per-process limit. The ambient tooling fleet already running in a session (MCP servers, language runtimes — tokio/V8/libuv worker pools, `mongod` / `chrome` / `prisma`-class MCP servers, local LLM servers) consumes a large FIXED fraction of this budget before

any project work even starts. **FORENSIC FACT (2026-07-07):** a host configured with `ulimit -u 4096` sat at ~3900–4005 threads from the ambient MCP fleet alone, leaving under 150 threads of headroom; dispatching a 4th Go-heavy subagent (the Go runtime sets `GOMAXPROCS` to `nproc`, spawning dozens of OS threads per invocation) hit `EAGAIN` / `failed to create new OS thread (errno=11)` and crashed tooling. Stopping three non-essential MCP servers (`mongod` / `chrome` / `prisma`) freed approximately 1345 threads (3827 → 2482), restoring headroom.

Mandate — treat thread-limit exhaustion as a §12 host-safety event. BEFORE scaling to N parallel subagents / processes, the agent MUST check thread headroom and treat thread-limit exhaustion as a §12 host-safety event that YIELDS UNCONDITIONALLY — the same standing as §12.6 (memory) and §12.7 (AOSP `-j` cap). This is an ORTHOGONAL exhaustion axis to §12.6's memory ceiling: a host can have abundant free RAM while its thread/process budget is fully exhausted, and vice versa — both MUST be checked independently before scaling parallelism.

Diagnosis (mandatory before scaling). `ulimit -u` (soft limit) + `ulimit -Hu` (hard limit) + the LIVE count `ps -L --no-headers -u "$USER" | wc -l` (threads, not just processes — `RLIMIT_NPROC` counts threads); headroom = soft limit minus live count. **Signals of exhaustion** (never guessed, always captured per §11.4.6): `EAGAIN`, `fork: retry: Resource temporarily unavailable`, `failed to create new OS thread`, or `cannot allocate memory` returned from a `fork` / `clone` /thread-spawn call — this is DISTINCT from OOM (memory can be abundant while the thread/process budget is exhausted; the diagnostic MUST NOT conflate the two). When headroom is low, the agent MUST SERIALIZE (run fewer concurrent subagents/processes) and avoid Go-heavy / `GOMAXPROCS=nproc`-class spikes — dispatching parallel work blind at near-limit headroom is forbidden; a subagent that hits `EAGAIN` mid-operation (e.g. mid-`git push`) risks incomplete/corrupted state (§9.2 data-safety risk).

Raising the limit — three techniques (cite the mechanism used, never guess which applies). (a) **No-restart, on a running process:** `prlimit --pid <PID> --nproc=<soft>:<hard>` (requires root — e.g. via `su -c`) to raise a HARD limit above the process's current hard ceiling; a running process's new children inherit the raised limit going forward. (b) **Persistent, system-wide:** `/etc/security/limits.conf` (`<user> hard nproc <N>` + a

matching `soft` line) or a systemd unit's `LimitNPROC=<N>`, taking effect on the NEXT login/session (not the current one). (c) **Self-raise, within the current session:** any process may raise its own SOFT limit up to its existing HARD limit via `ulimit -u <N>` where `N ≤ hard` — it can NEVER exceed the hard ceiling without root-assisted method (a). **Caveat (§11.4.6 — do not assume scope):** `su -c "ulimit -u N"` sets the limit ONLY inside that transient `su` subshell — it does NOT retroactively affect an already-running session; a running process keeps the `RLIMIT_NPROC` value captured at its own launch time.

Freeing threads — operator-authorized only (§11.4.122). Stopping non-essential MCP servers / background tooling frees the ambient fraction of the budget consumed before project work starts — but this REQUIRES explicit operator authorization per §11.4.122 (no-silent-removal); the agent MUST NOT autonomously kill the operator's tooling without asking first. **CAUTION — self-kill hazard (forensic FACT):** `pkill -f <pattern>` / `pgrep -f <pattern>` matches against the FULL COMMAND LINE, including the killer's own invocation if the search pattern string happens to appear in it — this can cause an unintended self-kill of the very process issuing the kill command. Any cmdline-pattern-based kill list MUST explicitly exclude the caller's own `$$` / `$PPID` (and the conductor's PID, when dispatched as a subagent) before executing.

Composes with §12 (host-session safety umbrella) / §12.6 (memory-budget ceiling — the ORTHOGONAL second exhaustion axis alongside this thread/process-count axis) / §12.7 (bare-metal AOSP `-j` cap — thread exhaustion is a DISTINCT failure mode from the memory-driven cap that section addresses) / §11.4.58 (parallel-development PWU pipeline — the concurrency this section bounds) / §11.4.103 (continuous parallel-stream working routine — the ≥3-background-stream target is bounded by available thread headroom, not merely memory) / §11.4.101 (autonomous-decision-over-blocking — safe, reversible decisions, e.g. serializing dispatch, are made without operator involvement) / §11.4.122 (no-silent-removal-of-existing-components — freeing ambient tooling threads requires operator authorization) / §9.2 (absolute data safety — a subagent crashed mid-git-operation by `EAGAIN` is a data-safety risk this section exists to prevent).

Classification: universal (§11.4.17) — `RLIMIT_NPROC` / `ulimit -u` is a Linux/POSIX kernel-level per-UID accounting mechanism, platform-agnostic and true on every host running this Constitution's parallel-subagent/multi-process disciplines; the consuming project supplies its concrete ambient-tooling roster and ulimit values

per §11.4.35. Propagation gate `CM-COVENANT-12-12-PROPAGATION` (literal anchor `12.12` present across the consumer fleet's CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md) + recommended gate `CM-NPROC-HEADROOM-CHECK` (parallel-dispatch orchestration checks `ulimit -u` + live thread count BEFORE scaling concurrency; a cmdline-pattern kill helper excludes the caller's own PID) + paired §1.1 meta-test mutation (strip the `12.12` literal → propagation gate FAILs; strip the headroom-check call before a parallel-dispatch scale-up OR remove the self-PID exclusion from a kill helper → `CM-NPROC-HEADROOM-CHECK` FAILs; gate-code = a separate tracked work item).

Canonical authority: constitution submodule `Constitution.md` §12.12. Non-compliance is a release blocker regardless of context. No escape hatch — no `--ignore-thread-limit`, `--dispatch-blind`, `--skip-headroom-check`, `--autonomous-tooling-kill`, `--assume-ulimit-scope` flag exists.

Appendix A — Mutation testing — academic and industrial foundations

The anti-bluff / paired-mutation policy in §1.1 is rooted in the **mutation testing** literature. References that the consuming project SHOULD cite when explaining its meta-test harness:

- Jia, Y. & Harman, M. (2011). *An Analysis and Survey of the Development of Mutation Testing*. IEEE Transactions on Software Engineering, 37(5).
- DeMillo, R.A., Lipton, R.J., Sayward, F.G. (1978). *Hints on Test Data Selection: Help for the Practicing Programmer*. IEEE Computer.
- Open-source mutation testers worth studying:
  - **PIT** (Java) — https://pitest.org/
  - **Stryker** (JS / C# / Scala) — https://stryker-mutator.io/
  - **Cosmic Ray** (Python) — https://github.com/sixty-north/cosmic-ray
  - **mutmut** (Python) — https://mutmut.readthedocs.io/
  - **mull** (LLVM-IR) — https://mull.readthedocs.io/

The §1.1 paired-mutation policy is a **lightweight** variant of mutation testing applied at the gate-assertion layer rather than at the production-code layer. It does not need a mutation framework; it needs ONE mutation per gate that proves the gate catches the break. This is computationally cheap (O(gates), not O(production code lines)) and produces a strong "bluff-immunity" guarantee.

Appendix B — Recursive inheritance & path-independence

Projects that include this constitution submodule MUST tolerate **arbitrary submodule depth**. Between the main project root and any consuming submodule there may be N intermediate levels. To locate this constitution submodule from any depth, every project SHOULD provide a helper that walks up parents until it finds `constitution/Constitution.md` (the file you are reading right now) OR follows `git rev-parse --show-superproject-working-tree` recursively. The constitution submodule ships `find_constitution.sh` for exactly this purpose; nested submodules can source it without knowing their own depth.